

## UNIDAD 1 — DEL NAVEGADOR AL SERVIDOR

# Clase 1 · Sistema de nombres de dominio

## Ejercicios de teoría — CLAVE DE CORRECCIÓN

Programación 3 — Tecnicatura Universitaria en Programación — UTN FRM

|                   |                                                                                                |
|-------------------|------------------------------------------------------------------------------------------------|
| Modalidad         | En grupos de dos o tres. Se responde con el apunte de la clase a mano.                         |
| Tiempo sugerido   | 50 a 60 minutos.                                                                               |
| Puntaje total     | 100 puntos. Partes A (20), B (10), C (30), D (30) y E (10).                                    |
| Qué se evalúa     | Que puedan nombrar el concepto que explica cada síntoma, no que recuerden comandos de memoria. |
| Uso de esta clave | Documento del docente. Cada respuesta cierra con la sección del apunte donde se justifica.     |

**Cómo usar esta clave.** Las respuestas están redactadas completas para que puedan entregarse tal cual como devolución. En las partes C, D y E no se busca la redacción textual sino que aparezcan los conceptos marcados en negrita; si el grupo nombra el concepto correcto con sus palabras, el ítem está bien.

### Parte A — Verdadero o falso, con justificación

*20 puntos (2 por ítem)*

Indicá si cada afirmación es verdadera o falsa. **La justificación es obligatoria:** una respuesta sin justificar no suma puntaje. Una línea alcanza.

- A1.** Si la resolución de un nombre falla, el log del servidor va a mostrar un error de conexión que permite diagnosticar el problema.

**FALSO.** La resolución de nombres es previa e independiente de la conexión: si falla la operación 1, las operaciones 2 a 5 nunca se ejecutan y la petición jamás se emite hacia el servidor. El log queda vacío. Un servidor con el log vacío no está "sin actividad": está ciego respecto de un fallo que ocurre antes de su puerta.

Apunte: §1.1 y §1.3

- A2.** Un registro CNAME puede usarse en el vértice de la zona (el dominio raíz pelado) siempre y cuando no se carguen otros registros con ese mismo nombre.

**FALSO.** El vértice de la zona requiere obligatoriamente registros NS y SOA, y un CNAME no puede coexistir con ningún otro registro en el mismo nombre. La condición "que no haya otros registros" es imposible de cumplir en el vértice. Por eso el dominio raíz siempre se apunta con un registro A.

Apunte: §1.6.1

- A3.** Un dominio recién registrado, que nunca fue consultado por nadie, resuelve casi de inmediato.

**VERDADERO.** No existe caché previa que expirar. Lo que demora no es la creación del dato sino la expiración de las copias viejas que los resolvers ya tenían guardadas; si nunca hubo consulta, no hay copia vieja.

Apunte: §1.8

- A4.** Los 13 servidores raíz del DNS son 13 máquinas físicas, y consultarlos desde Argentina implica cruzar el océano.

**FALSO.** Son 13 *identidades lógicas* (a.root-servers.net hasta m.root-servers.net), un límite heredado del tamaño máximo de una respuesta DNS sobre UDP en el diseño original. Cada identidad está replicada hoy en cientos de instancias físicas repartidas por el mundo mediante anycast, y hay instancias en América del Sur, incluida Argentina.

Apunte: §1.4.2

- A5.** Recibir NXDOMAIN y que la consulta agote el tiempo de espera son dos maneras de observar el mismo problema.

**FALSO.** NXDOMAIN es una respuesta: rápida, definitiva, alguien contestó "ese nombre no existe". Un timeout *no* es una respuesta: nadie contestó nada. Confundirlos lleva a diagnosticar filtrado donde en realidad hay un resolver caído o inalcanzable, es decir, a dar una explicación técnica perfectamente equivocada de un problema perfectamente real.

Apunte: §1.5 y §1.9.1

- A6.** Cargando un registro A con nombre \* queda cubierto todo el dominio, así que el registro con nombre @ es redundante.

**FALSO.** El comodín no cubre el dominio raíz: \*.tudominio.com no incluye a tudominio.com. Por eso el práctico pide dos registros: el comodín \* para los subdominios y el registro @ para el vértice.

Apunte: §1.12.1 y §1.14

- A7.** DNSSEC cifra las consultas DNS y evita que un intermediario pueda leerlas.

**FALSO.** DNSSEC *autentica, no cifra*: permite comprobar matemáticamente que la respuesta recibida es la que publicó el autoritativo, pero las consultas siguen viajando visibles para cualquier observador de la red. El cifrado del canal es lo que aportan DoT y DoH, que son otra cosa.

Apunte: §1.11.1 y §1.11.2

- A8.** Si cargás los registros en el panel del proveedor donde compraste el dominio, siempre van a tener efecto.

**FALSO.** Los registros se cargan donde apuntan los NS del dominio, no necesariamente donde se compró. Registrador y proveedor de DNS son funciones separables. Cargar los registros en el lugar equivocado no produce ningún error: simplemente no tiene efecto, y el panel te sigue mostrando los registros que cargaste aunque el mundo consulte otra zona.

Apunte: §1.7

- A9.** Bajar el TTL a 300 segundos justo antes de hacer un cambio garantiza que el cambio se vea en cinco minutos.

**FALSO.** La reducción del TTL también es un cambio, y las cachés la descubren recién cuando expira el TTL *viejo*. Por eso la técnica estándar es bajar el TTL con al menos un día de anticipación al cambio previsto, y restaurarlo una vez verificado el valor nuevo.

Apunte: §1.8

- A10.** Un nombre que existe en la zona pero no tiene ningún registro del tipo consultado devuelve NXDOMAIN.

**FALSO.** Devuelve **NOERROR con cero respuestas**. NOERROR significa que la consulta se procesó correctamente; no garantiza que haya registros en la respuesta. NXDOMAIN afirma algo más fuerte: que el nombre no existe.

Apunte: §1.5

---

## Parte B — Opción múltiple

10 puntos (1 por ítem)

Marcá **una sola** opción por ítem.

- B1.** ¿Qué código de respuesta devuelve un servidor que reconoce la consulta como válida pero decide por política no atenderla?
- a) NOERROR
  - b) SERVFAIL
  - c) NXDOMAIN
  - d) REFUSED

**d) REFUSED.** Es negativa por política, no por inexistencia del nombre (NXDOMAIN) ni por fallo de procesamiento (SERVFAIL).

Apunte: §1.5

**B2.** ¿Cuál de estos mecanismos transmite el nombre del sitio **en texto plano, antes** de que exista canal cifrado?

- a) La cabecera Host del pedido HTTP
- b) La extensión SNI de la negociación TLS
- c) El registro CAA de la zona
- d) Los glue records de la delegación

**b) La extensión SNI.** El servidor necesita saber qué certificado presentar *antes* de poder cifrar, y el certificado es justamente el material con el que se establece ese canal. Ese problema de huevo y gallina se resuelve enviando el nombre en claro. La cabecera Host, en cambio, viaja en la operación 4, ya dentro del canal cifrado.

Apunte: §1.3

**B3.** La cabecera Host del pedido HTTP existe fundamentalmente para que...

- a) ...el navegador sepa a qué IP conectarse
- b) ...un único servidor con una única IP pueda atender múltiples dominios distintos
- c) ...el resolver pueda cachear la respuesta
- d) ...Let's Encrypt valide la titularidad del dominio

**b).** Es el mecanismo del alojamiento virtual (*virtual hosting*): el proxy inverso lee ese valor y decide a qué servicio interno entregar la petición. Sin él, cada sitio web del mundo necesitaría una IP exclusiva, algo imposible dada la escasez de direcciones IPv4. Es lo que permite que un mismo VPS aloje `calculadora.tudominio.com` y `api.tudominio.com` con una sola IP.

Apunte: §1.3

**B4.** ¿Qué comando hay que correr **antes de tocar un solo registro** para saber dónde cargarlos?

- a) `nslookup tudominio.com`
- b) `nslookup -type=NS tudominio.com`
- c) `ipconfig /flushdns`
- d) `nslookup -type=CAA tudominio.com`

**b).** Los registros NS son la anotación, cargada en la zona del padre, de a quién se delegó la administración de la zona. Determinan cuál es el panel autoritativo, que es el único donde cargar registros tiene efecto.

Apunte: §1.7

**B5.** La razón por la que un CNAME no puede convivir con otros registros en el mismo nombre es que...

- a) ...las RFC lo prohibieron por razones históricas, sin fundamento técnico
- b) ...el CNAME afirma que ese nombre es otro nombre, y admitir datos propios simultáneos contradiría esa afirmación
- c) ...ocuparía demasiado espacio en la respuesta UDP de 512 bytes
- d) ...impediría la validación DNSSEC de la zona

**b).** La restricción es lógica, no caprichosa: la semántica del CNAME es de sustitución total. Si el nombre "es, a todos los efectos, aquel otro", tener además un A o un MX propios volvería ambigua toda consulta.

Apunte: §1.6.1

**B6.** El *stub resolver* que corre en la computadora del usuario...

- a) ...consulta a la raíz, después al TLD y después al autoritativo
- b) ...envía una consulta recursiva a un resolver y espera la respuesta final
- c) ...es autoritativo sobre los nombres del sistema operativo
- d) ...guarda una copia completa de la zona raíz

**b).** El stub sabe hacer una sola cosa: delegar todo el recorrido. Quien recorre la jerarquía con consultas iterativas es el resolver recursivo, y el usuario nunca consulta la jerarquía directamente.

Apunte: §1.4.3

**B7.** En la salida de `dig`, el indicador `aa` significa que...

- a) ...el resolver ofrece servicio recursivo
- b) ...la respuesta fue validada con DNSSEC
- c) ...quien responde es el servidor autoritativo de la zona: el dato viene del origen y no de una caché
- d) ...la respuesta vino truncada y hay que reintentar por TCP

**c).** AA = *Authoritative Answer*. Ojo con no confundirlo con `ra` (Recursion Available), con `ad` (authenticated data, que sí es DNSSEC) ni con `tc` (truncated).

Apunte: §1.5 y §1.11.1

**B8.** Un TTL de 14400 segundos, valor predeterminado en muchos paneles, equivale a...

- a) 5 minutos
- b) 1 hora
- c) 4 horas
- d) 24 horas

**c) 4 horas.** Es el valor que hace que cada error del práctico cueste media clase de espera; por eso durante la cursada se baja a 300 (5 minutos).

Apunte: §1.8 y §1.14

- B9.** El DNS resuelve perfecto, el servicio está sano y arranca, pero el certificado TLS nunca se emite y el log del proxy no dice nada útil. ¿Cuál es el primer sospechoso?
- a) Un registro CAA que no incluye a la autoridad certificadora
  - b) Un TTL demasiado bajo
  - c) La falta de un registro AAAA
  - d) Caché negativa en el resolver del proveedor

**a).** Toda autoridad certificadora está obligada a consultar los CAA del nombre antes de emitir y a negarse si no figura en la lista. El síntoma es exactamente ese: todo resuelve, todo funciona, y el certificado no llega. Los dominios recién registrados no suelen traer CAA, así que el caso aparece sobre todo con dominios reutilizados de un proyecto viejo o del trabajo.

**Apunte:** §1.6.2

- B10.** La diferencia operativa más relevante entre DoT y DoH es que...
- a) DoH autentica las respuestas y DoT no
  - b) DoT viaja por el puerto dedicado 853 y es identificable y bloqueable como tráfico DNS, mientras que DoH va por el 443 y bloquearlo implicaría bloquear HTTPS entero
  - c) DoT elimina al intermediario y DoH no
  - d) DoH solo funciona con dominios firmados con DNSSEC

**b).** Los dos cifran el mismo tramo —del equipo al resolver— con distinto empaque. Ninguno autentica el dato (eso es DNSSEC) y ninguno elimina al intermediario: redistribuyen la confianza hacia el resolver que uno elige.

**Apunte:** §1.11.2

---

## Parte C — Desarrollo conceptual

30 puntos (3 por ítem)

Respondé con precisión y brevedad. Se evalúa que uses el vocabulario técnico del capítulo, no la extensión de la respuesta.

- C1.** HOSTS.TXT fue la primera implementación de la "agenda de contactos" de la red. Explicá por qué colapsó (nombrá al menos tres causas) y señalá qué vestigio de ese archivo sigue vivo hoy en cualquier sistema operativo.

Era un único archivo de texto mantenido a mano por el Network Information Center del Stanford Research Institute y descargado periódicamente por FTP por todas las máquinas de ARPANET. Colapsó porque: (1) el archivo crecía sin límite; (2) el tráfico para descargarlo se multiplicaba con cada nodo nuevo; (3) los nombres empezaron a colisionar, porque no había autoridad que garantizara unicidad más allá de una oficina; y (4) la copia de cada máquina estaba siempre desactualizada respecto de la realidad.

El vestigio: `/etc/hosts` en Unix y `C:\Windows\System32\drivers\etc\hosts` en Windows, que además siguen teniendo prioridad sobre cualquier consulta a la red.

**Apunte:** §1.2

- C2.** Enumerá las tres decisiones de diseño con las que Mockapetris resolvió el problema en 1983 y, para cada una, indicá una consecuencia observable en este práctico.

**1) Distribuir la autoridad, no los datos.** Nadie posee la base completa; cada organización es autoritativa sobre su porción y la unicidad queda garantizada por construcción. Consecuencia: los registros se cargan donde apuntan los NS, y cargarlos en otro panel no hace nada (§1.7).

**2) Aceptar la coherencia eventual a cambio de escala.** Cada respuesta lleva un TTL durante el cual puede copiarse y reutilizarse. Consecuencia: un cambio tarda en verse y "a un compañero le anda y a otro no" (§1.8).

**3) Un esquema de datos extensible: registros de recursos tipados.** Consecuencia: el mismo nombre lleva a la vez A, TXT, CAA, MX; y el protocolo sobrevivió cuatro décadas —cuando apareció IPv6 se agregó el tipo AAAA sin tocar nada (§1.6).

Apunte: §1.2

- C3.** Diferenciá **dominio** de **zona**. Usá `com` y `tudominio.com` como ejemplo.

Un **dominio** es un subárbol completo: el dominio `com` incluye, en sentido amplio, todo lo que termina en `.com`. Una **zona** es la porción del árbol que una organización administra efectivamente, y termina exactamente donde empieza una delegación hacia otra organización.

El operador de `com` no administra los registros de `tudominio.com`: su zona contiene apenas la anotación de a quién le delegó ese subárbol. Los registros concretos viven en la zona delegada, en el panel del proveedor que el titular eligió. La delegación es el acto administrativo que corta el árbol en zonas.

Apunte: §1.4.1 y §1.7

- C4.** Un servidor autoritativo y un resolver recursivo cumplen funciones opuestas. Explicá la diferencia y decí con cuál de los dos habla tu computadora.

El **autoritativo** posee los datos de su zona y responde solo sobre ella, sin salir a preguntar nada. El **recursivo** no posee ningún dato propio, pero sabe encontrar cualquiera: recorre la jerarquía con consultas iterativas (raíz » TLD » autoritativo), consolida el recorrido en una única respuesta, la guarda en su caché y la devuelve.

Tu computadora habla con el **resolver recursivo** —el que le asigna el proveedor por DHCP, o uno público como 8.8.8.8 / 1.1.1.1 / 9.9.9.9—. Nunca consulta la jerarquía directamente.

Apunte: §1.4.3

- C5.** "Hay que esperar a que propague." Explicá por qué la expresión es engañosa y describí qué ocurre realmente cuando se modifica un registro.

Sugiere que la información se difunde activamente por la red, como una onda que avanza de servidor en servidor. El DNS no tiene ningún mecanismo de difusión: nadie notifica a nadie. Los servidores autoritativos se actualizan de forma prácticamente inmediata; lo que demora es la expiración de las copias en caché que los resolvers del mundo obtuvieron antes del cambio.

Cada resolver descubre el valor nuevo en un momento distinto: exactamente cuando su copia vieja expira y una consulta lo obliga a volver al autoritativo. La "propagación" no es un proceso: es la suma de millones de expiraciones independientes.

Apunte: §1.8

- C6.** ¿Qué es la **caché negativa**, qué registro gobierna su duración y qué mala práctica de laboratorio castiga?

Los resolvers no solo recuerdan las respuestas afirmativas: también recuerdan los NXDOMAIN, durante un tiempo gobernado por el último parámetro del registro **SOA** de la zona (RFC 2308).

Castiga consultar un nombre *antes* de haber creado su registro: el resolver memoriza que no existe y seguirá contestando NXDOMAIN durante varios minutos aunque el registro ya esté cargado correctamente. Consultar "para ver si ya está" antes de tiempo puede, literalmente, demorar el momento en que se lo ve.

Apunte: §1.8

- C7.** Un compañero propone: "el filtrado se arregla poniendo 8.8.8.8 como DNS en la máquina". Explicá por qué eso resuelve el síntoma pero no es una solución, y cuál es la única variable que está del lado de quien publica.

Cambiar el resolver arregla el acceso *en esa máquina*, pero la aplicación seguiría siendo inaccesible para cualquier visitante cuyo proveedor aplique el mismo filtrado. El error de razonamiento es confundir la propia experiencia de acceso con la accesibilidad del servicio: quien publica no controla los resolvers de sus visitantes.

La única variable del lado del publicador es la **reputación del nombre**, y un dominio propio delegado a una zona limpia es la forma de controlarla. Por eso el dominio propio no es un adorno ni un paso opcional: es el único mecanismo que garantiza alcanzabilidad con independencia del resolver de cada visitante.

Apunte: §1.9.3



- C8.** Completá el cuadro comparando **registro DNS comodín** y **certificado TLS comodín**: qué es cada uno, para qué sirve, qué necesita y si se usa en este módulo.

**Registro DNS comodín:** es un registro A con nombre \*; sirve para que cualquier subdominio resuelva; se carga en el panel DNS y no necesita nada más; dificultad trivial; **sí** se usa en este módulo.

**Certificado TLS comodín:** es un certificado válido para \*.dominio.com; sirve para que cualquier subdominio tenga HTTPS; requiere el desafío DNS-01, es decir un token de API del proveedor DNS configurado en Traefik; dificultad considerable; **no** se usa en este módulo, porque cada servicio publicado en Easypanel pide su propio certificado individual por el desafío HTTP-01, que es automático.

*Asimetría que conviene notar:* el comodín DNS cubre nombres de varios niveles (a.b.tudominio.com), mientras que el certificado comodín cubre un solo nivel de etiqueta.

Apunte: §1.12

- C9.** Distinguí los tres roles comerciales que el lenguaje cotidiano mezcla —*registry*, *registrar* y *registrant*— y explicá qué consecuencia práctica tiene que estén separados.

**Registro (registry):** la organización que opera un TLD completo — Verisign opera .com, NIC Argentina opera .ar.

**Registrador (registrar):** la empresa minorista donde se compra el dominio (Namecheap, name.com); inscribe el dominio ante el registro y declara sus servidores de nombres.

**Titular (registrant):** quien tiene los derechos sobre el nombre.

Consecuencia: registrador y proveedor de DNS son funciones separables. Comprar el dominio en un lugar y administrar sus registros en otro es perfectamente normal, y son los registros NS los que expresan esa elección. De ahí la regla: los registros se cargan en el panel al que apuntan los NS.

Apunte: §1.7

- C10.** Las cuatro comprobaciones de cierre del capítulo no son cuatro variantes de la misma prueba. Indicá qué valida cada una.

1. `nslookup -type=NS tudominio.com` » valida la **delegación**: que los NS sean los del proveedor donde efectivamente se cargaron los registros (§1.7).

2. `nslookup tudominio.com` » valida el **registro de vértice** (@), que el comodín no cubre.

3. `nslookup loquesea.tudominio.com` » valida la **semántica del comodín**: ese nombre no fue cargado en ninguna parte y sin embargo debe resolver (§1.12).

4. `nslookup loquesea.tudominio.com 8.8.8.8` » valida la **coherencia entre resolvers independientes**, que es la única defensa contra el efecto "a mí me anda" (§1.10).

Apunte: §1.15

## Parte D — Casos de diagnóstico

30 puntos (5 por caso)

Para cada caso indicá: **(a)** el diagnóstico, **(b)** el comando o la evidencia con la que lo confirmarías y **(c)** la corrección. Nombrá el concepto del capítulo que explica el síntoma.

- D1.** Un grupo compró el dominio en Namecheap, cargó ahí el registro A comodín, lo ve en la tabla *Advanced DNS* con todos los datos correctos, y el dominio no resuelve desde ningún lado ni después de tres horas.

**(a) Diagnóstico:** los nameservers están delegados a otro proveedor (típicamente Cloudflare, porque alguien tocó esa opción). Namecheap no es autoritativo para esa zona, así que el panel muestra los registros que el grupo cargó, no los que el mundo consulta. Es el error número uno del capítulo.

**(b) Evidencia:** `nslookup -type=NS tudominio.com`. Si devuelve `xxx.ns.cloudflare.com`, la zona la administra Cloudflare; si devuelve `dns1.registrar-servers.com`, la administra Namecheap.

**(c) Corrección:** cargar los registros en el panel del proveedor al que apuntan los NS, o bien devolver los NS a Namecheap.

**Concepto:** delegación — los registros se cargan donde apuntan los NS.

**Apunte:** §1.7

- D2.** Una estudiante corre las cuatro consultas de control desde su casa y obtiene: `google.com` **resuelve**; `easypanel.host` **no resuelve**; el nombre autogenerado **no resuelve**; el mismo nombre contra `8.8.8.8` **resuelve**. ¿Qué puede afirmar y qué no?

**(a) Diagnóstico: filtrado confirmado.** Es la única fila de la tabla de §1.9.2 que autoriza a afirmarlo. La primera consulta prueba que el resolver está vivo (descarta "caído"); la segunda muestra que el bloqueo alcanza a la *zona entera*, no al nombre puntual; la cuarta aísla la variable: con otro instrumento, el mismo nombre resuelve.

**(b) Evidencia:** las cuatro consultas —el control incluido— más `ipconfig /all` para identificar qué resolver le asignó la red.

**(c) Qué NO puede afirmar:** que el problema esté en el dominio o en el servidor. El bloqueo está en el resolver, y puede deberse a listas de reputación (esas zonas se usan mucho para phishing y C2, y bloquear la zona entera le sale barato al operador) o a protección contra DNS rebinding. Tampoco puede afirmar que sea permanente ni que alcance a todo el proveedor: las listas cambian, y un mismo operador puede filtrar en su servicio hogareño y no en su red móvil.

**Corrección real:** dominio propio; cambiar el resolver propio no soluciona nada para los visitantes.

**Apunte:** §1.9.2 y §1.9.3

- D3.** Un grupo consulta su subdominio y el resolver contesta que no existe `calculadora.tudominio.me.tudominio.me`.

**(a) Diagnóstico:** escribieron el nombre completo en el campo *Nombre / Host* del panel. El panel agrega el dominio solo, así que el nombre quedó concatenado dos veces y apunta a un nombre que no existe y no va a resolver nunca.

**(b) Evidencia:** el propio mensaje de la consulta, y la columna *Host* de la tabla del panel.

**(c) Corrección:** dejar en el campo Nombre solamente `*`, `@` o `api`, según corresponda.

**Concepto:** el nombre del registro es relativo a la zona; el FQDN lo compone el panel.

**Apunte:** §1.14 y §1.16

- D4.** `calculadora.tudominio.me` y `api.tudominio.me` resuelven perfecto a la IP del VPS, pero `tudominio.me` pelado no resuelve.

(a) **Diagnóstico:** falta el registro del vértice. El comodín `*` no cubre el dominio raíz: `*.tudominio.me` no incluye a `tudominio.me`.  
(b) **Evidencia:** `nslookup tudominio.me` frente a `nslookup loquesea.tudominio.me`: el segundo anda y el primero no.  
(c) **Corrección:** agregar un registro **A** con nombre `@` apuntando a la IP del VPS. Tiene que ser **A** y no **CNAME**, porque el vértice requiere **NS** y **SOA**, con los que un **CNAME** no puede convivir.  
**Concepto:** alcance del comodín + restricción del **CNAME** en el vértice.

Apunte: §1.12.1 y §1.6.1

- D5.** Un grupo consultó `api.tudominio.me` mientras discutía cómo cargar el registro. Diez minutos después crea el registro correctamente, vuelve a consultar y sigue recibiendo **NXDOMAIN**. Verifican el panel tres veces y el registro está bien.

(a) **Diagnóstico:** **caché negativa** (RFC 2308). El resolver memorizó el **NXDOMAIN** de la consulta prematura y lo va a seguir contestando durante el tiempo que fija el último parámetro del **SOA** de la zona, aunque el registro ya exista.  
(b) **Evidencia:** consultar el mismo nombre contra un resolver distinto (`nslookup api.tudominio.me 8.8.8.8`) o directamente contra el autoritativo: si ahí resuelve, el dato está bien y el problema es la copia negativa.  
(c) **Corrección:** esperar el **TTL** negativo, o trabajar contra otro resolver mientras tanto. `ipconfig /flushdns` limpia las capas locales pero no la del proveedor, que no se puede vaciar.  
**Moraleja:** consultar "para ver si ya está" antes de crear el registro demora el momento en que se lo ve.

Apunte: §1.8 y §1.8.1

- D6.** En la mesa, un estudiante dice que el sitio anda y tres compañeros dicen que no. Están todos en la misma aula. Explicá cómo puede ser que los cuatro estén diciendo la verdad y describí el procedimiento para verificarlo en serio.

(a) **Diagnóstico:** están viendo cachés distintas y, eventualmente, resolvers distintos. Una consulta atraviesa hasta cuatro cachés (navegador, sistema operativo, router doméstico, resolver del proveedor), cada una con su propio reloj de expiración, y no las comparten todas. Además, un navegador con **DoH** activo hacia `1.1.1.1` esquivo el resolver de la red y puede dar un resultado distinto en el mismo equipo. "A mí me anda" es una muestra de tamaño uno, tomada desde un solo resolver, con una sola caché: no es una verificación.  
(b) **Procedimiento:** primero `ipconfig /flushdns` (y, si hace falta, limpiar la caché del navegador en `chrome://net-internals/#dns`) y reintentar; después consultar contra un resolver público explícito; y finalmente **dnschecker.org**, que consulta decenas de resolvers distribuidos por el mundo y muestra el resultado en un mapa.  
(c) **Lectura:** que uno o dos resolvers del mapa no devuelvan nada no prueba que haya un error: puede ser filtrado, una caché negativa previa a la creación del registro, o ese resolver fuera de servicio en ese instante — la herramienta no permite distinguirlos.

Apunte: §1.8.1 y §1.10

## Parte E — Lectura de salidas reales

10 puntos

Leer la salida de una herramienta es leer el protocolo. Respondé mirando el bloque de cada ítem.

E1.

```
; <<>> DiG 9.18.18 <<>> tudominio.me
;; ->>HEADER<- opcode: QUERY, status: NOERROR, id: 41277
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; ANSWER SECTION:
tudominio.me.      287      IN      A      72.61.33.27

;; SERVER: 8.8.8.8#53(8.8.8.8) (UDP)
```

**(4 puntos)** Indicá: (i) el RCODE y qué significa; (ii) quién respondió; (iii) si el dato vino del servidor autoritativo y cómo lo sabés; (iv) qué TTL tenía originalmente el registro y por qué el número que se ve no es redondo.

**(i)** status: NOERROR: la consulta se procesó correctamente. Acá además hay ANSWER: 1, así que sí hay dato; NOERROR por sí solo no lo garantizaría.  
**(ii)** Respondió 8.8.8.8, el resolver público de Google (línea SERVER).  
**(iii)** **No** vino del autoritativo: entre las flags aparecen qr rd ra pero **no** aparece aa. La respuesta salió de la caché del resolver.  
**(iv)** El TTL configurado en la zona es **300** (5 minutos, el valor que pide el práctico). Se ven **287** porque el TTL que devuelve un resolver es el *remanente*: la copia lleva 13 segundos en su caché y el contador va bajando hasta expirar.

Apunte: §1.5, §1.8 y §1.10

E2.

```
> nslookup -type=NS tudominio.me

Respuesta no autoritativa:
tudominio.me      nameserver = dns1.registrar-servers.com
tudominio.me      nameserver = dns2.registrar-servers.com
```

**(3 puntos)** ¿En qué panel hay que cargar los registros y por qué? ¿Qué habría que concluir si la salida hubiera sido ns1.dns-parking.com?

Los NS apuntan a registrar-servers.com, que es **Namecheap**: ahí hay que cargar los registros, porque es el proveedor al que la zona fue delegada y por lo tanto el único autoritativo. Cargarlos en cualquier otro panel no da error, simplemente no tiene efecto.

Si hubiera devuelto ns1.dns-parking.com, la zona la administra **Hostinger** y los registros van en el panel de Hostinger. (xxx.ns.cloudflare.com » Cloudflare.)

Detalle: "Respuesta no autoritativa" es normal — significa que contestó un resolver desde su caché, no el autoritativo. No es un error.

Apunte: §1.7

**E3.**

```
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 9032
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 0

--- versus ---

;; connection timed out; no servers could be reached
```

**(3 puntos)** Las dos salidas se ven parecidas y no son lo mismo. Explicá la diferencia y qué conclusión distinta habilita cada una.

**Arriba** hay una *respuesta*: alguien contestó, rápido y de forma definitiva, que ese nombre no existe en la zona autoritativa (o eso afirma quien responde). El resolver está vivo y funcionando. Es compatible con un nombre mal escrito, con un registro que todavía no se cargó, o con un resolver que filtra.

**Abajo** *no hay respuesta*: la consulta agotó el tiempo de espera y se reintentó varias veces antes de rendirse. El resolver está caído o es inalcanzable. Acá no se puede concluir absolutamente nada sobre el nombre consultado.

Por eso el procedimiento de diagnóstico empieza con la prueba de control (nslookup google.com): antes de medir el fenómeno hay que verificar el instrumento. Tomar un fallo intermitente del resolver por filtrado lleva a una explicación técnica perfectamente equivocada de un problema perfectamente real.

**Apunte:** §1.5 y §1.9.1

## Cierre — una para pensar (sin puntaje)

El capítulo dice que DNSSEC, DoT y DoH **redistribuyen la confianza, no la eliminan**: la pregunta "¿quién ve mis consultas DNS?" nunca tiene respuesta "nadie", tiene respuesta "el que vos elegiste como resolver".

Discutan en el grupo: si eligen 8.8.8.8 para esquivar el filtrado de su proveedor, ¿qué le están dando a Google que antes tenía su proveedor? ¿Y qué pasa con el argumento "no tengo nada que ocultar" cuando lo que se cede no es el contenido sino la lista completa de todo lo que uno visita?

No hay respuesta única — el objetivo es que quede claro que **elegir resolver es elegir en quién confiar**, y que en ningún escenario el intermediario desaparece. Un buen cierre reconoce que: (1) sin DoH, el proveedor ve las consultas; con DoH hacia un resolver público, las ve ese resolver; (2) la metadata de navegación —qué dominios, cuándo, con qué frecuencia— es un perfil muy completo aunque el contenido esté cifrado; y (3) del lado de quien *publica* un servicio, nada de esto está bajo su control: lo único que controla es la reputación de su nombre (§1.9.3).